SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I **RenugaDevi B (192311111)***(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
RenugaDevi B

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1. Errors in Equivalence Class Partitioning and Boundary Value Analysis

Introduction

Equivalence Class Partitioning (ECP) divides input data into valid and invalid groups. Boundary Value Analysis (BVA) tests values at and around the limits. Incorrect classification and missing boundary tests can cause important defects to remain undetected.

1. Errors Identified

The tester incorrectly combines valid and invalid inputs in the same equivalence class. This violates the principle that inputs in one class should be expected to behave similarly. The tester also ignores boundary values, where validation errors commonly occur.

2. Root Cause Analysis

These mistakes may occur because the tester has not identified the input constraints or misunderstood the application's validation rules. Testing only ordinary inputs may fail to detect errors near minimum and maximum limits.

3. Corrected Test Cases

Assume an online shopping system accepts product quantities from 1 to 100.

Equivalence Class	Input	Expected Result
Invalid	0	Reject
Valid	1–100	Accept
Invalid	Negative quantity	Reject
Invalid	Above 100	Reject
Invalid	Text instead of a number	Reject

Boundary tests should include **0, 1, 2, 99, 100, and 101**.

4. Justification

The corrected approach separates inputs according to expected behavior and tests values near both limits. It improves defect detection by identifying incorrect validation, off-by-one errors, and improper acceptance of invalid quantities.

Conclusion

Correct ECP and BVA provide systematic input coverage and make the online shopping system more reliable.

2. Errors in Random Testing of a Banking Application

Introduction

Random testing selects inputs without following a structured test design. Although it can discover unexpected defects, relying on it alone does not guarantee adequate coverage of critical banking operations.

1. Errors Identified

The QA engineer assumes that random testing provides high coverage without measuring executed conditions or tested boundaries. Transaction limits and important combinations of inputs may remain untested.

2. Root Cause Analysis

Randomly selected inputs may concentrate on ordinary transactions rather than values near transaction limits. The tester may also fail to define coverage criteria, expected results, and important risk areas.

For example, a withdrawal limit of ₹20,000 may not be tested at exactly ₹20,000 or immediately above it.

3. Corrected Testing Approach

Boundary Value Analysis: Test withdrawals of ₹19,999, ₹20,000, and ₹20,001.

Equivalence Partitioning: Divide withdrawal amounts into valid amounts, zero amounts, negative amounts, and amounts exceeding the permitted limit.

Decision Table Testing: Test combinations of sufficient balance, valid account status, and permitted withdrawal amount.

State Transition Testing: Verify account and transaction states during successful, failed, and reversed transactions.

4. Justification

Structured techniques ensure that important input classes, limits, and transaction conditions are tested systematically. Random testing can still supplement these techniques by discovering unexpected input combinations.

Conclusion

Random testing alone is inadequate for critical banking operations. Combining structured black box techniques with risk-based testing improves coverage, reliability, and defect detection.

3. Errors in Control Flow Graph Construction and Path Testing

Introduction

White box testing examines the internal logic of a program. A Control Flow Graph (CFG) represents statements, decisions, loops, and execution paths. An incomplete CFG results in inadequate path coverage.

1. Errors Identified

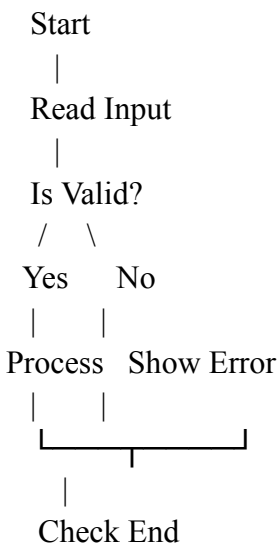
The developer omits decision nodes and loops from the CFG. As a result, some execution paths are not identified, and test cases cannot cover all important branches.

2. Root Cause Analysis

The developer may have misunderstood conditional statements or failed to represent both true and false outcomes. Loops may also be overlooked, preventing tests of zero, one, and multiple iterations.

3. Corrected Control Flow Representation

Consider this example:



For programs containing loops, the CFG must include an edge from the loop body back to the loop condition and an exit edge when the condition becomes false.

4. Independent Paths

For the illustrated decision, the independent paths are:

- Path 1: Start → Read Input → Valid → Process → End.
- Path 2: Start → Read Input → Invalid → Show Error → End.

Additional paths must be included for every other decision and loop in the actual program.

Cyclomatic complexity can be calculated using:

[
V(G)=E-N+2P
]

where (E) is the number of edges, (N) is the number of nodes, and (P) is the number of connected components.

5. Justification

A complete CFG identifies missing branches and independent paths. Testing these paths improves branch coverage and helps detect logical errors.

Conclusion

Correct CFG construction is essential for effective path testing. Every decision and loop must be represented before designing the final test suite.

4. Errors in Relying Only on Statement Coverage

Introduction

Statement coverage measures whether every executable statement has been executed at least once. However, 100% statement coverage does not guarantee that all decision outcomes have been tested.

1. Errors Identified

The testing team considers complete statement coverage sufficient. Consequently, logical errors in conditional statements may remain undetected because some branches have not been executed.

2. Root Cause Analysis

The main cause is the use of an insufficient coverage criterion. A test case may execute a statement through one decision outcome without testing the alternative outcome.

For example:

```
if (age >= 18)
   print("Eligible");
else
   print("Not eligible");
```

Testing only age = 20 executes the eligible branch but does not test the other branch.

3. Corrected Testing Strategy

Test Case	Input	Expected Result
TC1	Age = 20	Eligible
TC2	Age = 16	Not eligible
TC3	Age = 18	Eligible

The revised strategy should include:

- **Branch Coverage:** Test both true and false outcomes.
- **Condition Coverage:** Test each Boolean condition as true and false.
- **Path Testing:** Test independent execution paths.

4. Justification

Branch coverage detects untested decision outcomes. Condition coverage is particularly useful for complex Boolean expressions, while path testing examines combinations of execution steps.

Conclusion

Statement coverage alone is insufficient for detecting logical defects. Combining statement, branch, and condition coverage provides stronger testing adequacy.

5. Errors in Equivalence Class Partitioning for Age Validation

Introduction

Equivalence Class Partitioning divides input values into groups expected to behave similarly. For an age-validation field accepting ages from 0 to 100, the tester creates only two classes: negative and positive numbers.

1. Errors Identified

The partitioning is incorrect because it groups all positive numbers together, even though some are valid and others exceed the permitted maximum. It also fails to distinguish valid boundary values from invalid ones.

2. Root Cause Analysis

The tester has not considered the complete valid range or the system's input constraints. Boundary Value Analysis is also missing, so errors at ages 0 and 100 may remain undetected.

3. Corrected Equivalence Classes

Assuming the permitted integer age range is 0–100:

Class	Input Range	Expected Result
Invalid	Below 0	Reject
Valid	0–100	Accept
Invalid	Above 100	Reject
Invalid	Non-numeric input	Reject

4. Boundary Value Analysis

Test the following values:

Input	Expected Result
−1	Reject
0	Accept
1	Accept
99	Accept
100	Accept
101	Reject

The tests assume that both endpoints are valid.

5. Justification

The corrected partitions distinguish valid and invalid values clearly. Boundary testing detects off-by-one errors and incorrect range comparisons.

Conclusion

Proper equivalence partitioning and boundary value analysis improve age validation and prevent incorrect acceptance or rejection of inputs.

6. Errors in Condition Coverage for a Complex Boolean Expression

Introduction

Condition coverage checks whether each individual Boolean condition evaluates to both true and false at least once. For a compound expression such as if (A && B), testing only the overall decision result is insufficient.

1. Errors Identified

The developer may incorrectly assume that executing the if statement is enough to achieve condition coverage. However, both A and B must independently take true and false values.

2. Root Cause Analysis

The problem arises from confusing decision coverage with condition coverage. Because the && operator uses short-circuit evaluation, B may not be evaluated when A is false.

3. Corrected Test Cases

Test	A	B	Expected Decision
TC1	True	True	True
TC2	True	False	False
TC3	False	True	False

These tests ensure that both conditions take true and false values. However, in languages with short-circuit evaluation, B may not actually be evaluated in TC3.

4. Improved Testing Strategy

The tester should verify condition coverage and decision coverage separately. Modified Condition/Decision Coverage (MC/DC) can provide stronger assurance by demonstrating that each condition independently affects the overall decision.

5. Justification

The corrected strategy identifies defects caused by incorrect Boolean conditions and missing decision outcomes. It is particularly useful for safety-critical or complex decision logic.

Conclusion

Condition coverage must be measured independently from statement coverage. Testing individual Boolean conditions improves the detection of logical errors.

7. Errors in Decision Table Testing for a Discount System

Introduction

Decision Table Testing verifies system behavior for different combinations of conditions and their corresponding actions. In a discount system, the conditions may include premium membership and coupon application.

1. Errors Identified

The testing team omits several combinations of membership and coupon status. As a result, some discount rules remain untested, causing incorrect price calculations.

2. Root Cause Analysis

The likely causes include incomplete identification of conditions, missing decision-table rules, and failure to define expected results for every combination.

3. Corrected Decision Table

Assume the system gives a 10% premium-member discount and a 5% coupon discount. For this example, the discounts are additive and calculated on the original price.

Rule	Premium Member	Coupon Applied	Expected Discount
R1	Yes	Yes	15%
R2	Yes	No	10%
R3	No	Yes	5%
R4	No	No	0%

The system should calculate the final price using:

$$
\text{Final Price}=\text{Original Price}-\text{Discount}
$$

For an original price of ₹1,000, a premium member using a coupon receives a ₹150 discount, resulting in a final price of ₹850.

4. Improved Testing Strategy

Each rule should be tested with a suitable input. The tester should verify the discount amount, final price, and handling of invalid or expired coupons.

If the business rules specify that discounts cannot be combined, the expected results must be changed accordingly.

5. Justification

The revised decision table covers all four combinations of the two Boolean conditions. It prevents missing-rule defects and improves the accuracy of discount calculations.

Conclusion

A complete decision table ensures that every relevant combination is tested and that the system applies the correct pricing rules.

8. Errors in Loop Testing Strategy

Introduction

Loop testing checks whether loops execute correctly for different numbers of iterations. The tester checks only the first and maximum iterations, ignoring zero iterations and values beyond the permitted maximum.

1. Errors Identified

Testing only (n=1) and (n=\text{max}) misses important cases such as (n=0) and (n=\text{max}+1). Therefore, loop entry conditions, exit conditions, and limit validation may contain undetected defects.

2. Root Cause Analysis

The tester assumes that checking the smallest positive and maximum iteration counts is sufficient. This ignores situations in which the loop should not execute or the input exceeds the permitted limit.

3. Corrected Test Cases

Assume a loop permits a maximum of 10 iterations.

Iterations	Expected Behavior
(n=0)	Loop does not execute
(n=1)	Executes once
(n=2)	Executes twice
(n=9)	Executes nine times
(n=10)	Executes ten times
(n=11)	Rejects input or handles it according to the specification

4. Testing Nested Loops

For nested loops, the tester should:

- Test the inner loop independently where possible.
- Test the outer loop with zero, one, and multiple iterations.
- Check combinations of inner and outer loop counts.
- Test boundary values and exit conditions.
- Avoid excessively large combinations by selecting representative paths.

5. Justification

The corrected approach identifies off-by-one errors, incorrect termination conditions, and unexpected behavior when the loop receives invalid iteration counts.

Conclusion

Loop testing must cover zero, one, typical, maximum, and beyond-maximum iterations where applicable. Testing nested loops systematically improves execution correctness and path integrity.

9. Errors in Using Error Guessing Alone for a Safety-Critical Braking System

Introduction

Error Guessing is a testing technique that relies on a tester's experience to predict likely defects. Although useful for discovering unexpected failures, it is insufficient as the only technique for a safety-critical braking system.

1. Errors Identified

The engineer depends entirely on experience-based testing and ignores structured black box techniques. This can leave critical boundary conditions and braking-state transitions untested.

2. Root Cause Analysis

The main causes are the absence of systematic test design, insufficient risk analysis, and overconfidence in the tester's ability to predict defects.

For example, a braking system may behave correctly under normal conditions but fail when sensor values approach a critical threshold.

3. Corrected Testing Strategy

Boundary Value Analysis (BVA): Test sensor inputs immediately below, at, and above specified braking thresholds.

For example, if a braking controller activates at a sensor value of 50, test 49, 50, and 51, according to the defined requirements.

State Transition Testing: Test transitions between states such as:

- Normal operation → Braking activated.
- Braking activated → Vehicle stopped.
- Normal operation → Sensor fault.
- Sensor fault → Safe response.

Error Guessing: Use experience to add tests for unexpected sensor readings, intermittent faults, and unusual operating conditions.

4. Justification

BVA systematically examines critical thresholds, while state transition testing checks whether the system responds correctly to changing operating conditions. Error guessing supplements these techniques by targeting additional risks.

Because braking is safety-critical, testing must also include hazard analysis, fault injection, fail-safe behavior verification, and applicable safety standards.

Conclusion

Relying only on error guessing creates significant testing gaps. Combining structured testing techniques with risk-based safety verification improves defect detection and provides stronger safety assurance.

10. Errors in a Control Flow Graph with Missing Else Branches

Introduction

A Control Flow Graph represents the statements and decision paths of a program. When a tester omits else branches from nested if statements, the graph becomes incomplete and error-handling paths may remain untested.

1. Errors Identified

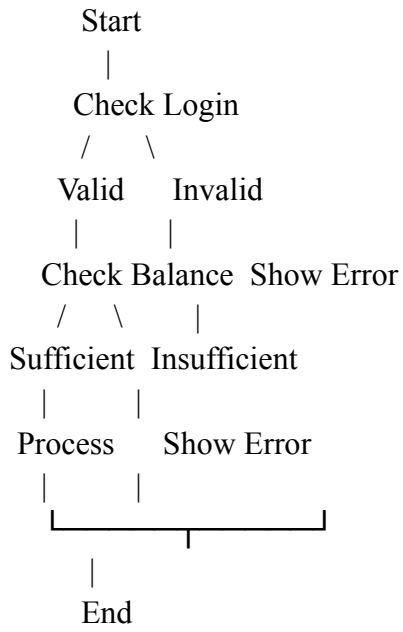
The CFG does not represent all possible decision outcomes. Although statement coverage may reach 100%, the false branches of nested conditions may never execute.

2. Root Cause Analysis

The tester may have drawn the graph only from the successful execution path. Missing else branches result in incomplete path identification and incorrect assumptions about coverage.

3. Corrected Control Flow Graph

Consider the following example:



The corrected graph represents the successful login path, insufficient-balance path, and invalid-login path.

4. Independent Paths

- **Path 1:** Start → Valid Login → Sufficient Balance → Process → End.
- **Path 2:** Start → Valid Login → Insufficient Balance → Show Error → End.
- **Path 3:** Start → Invalid Login → Show Error → End.

Each path should be executed using appropriate test data.

5. Improved Testing Strategy

The tester should identify every decision node, represent both true and false outcomes, calculate cyclomatic complexity, and design test cases for the independent paths. Branch coverage should then be measured to confirm that every decision outcome has been tested.

6. Justification

The corrected CFG reveals error-handling paths that statement coverage alone may miss. Testing all identified branches improves decision coverage and helps detect defects in validation and error handling.

Conclusion

A complete CFG must represent every decision outcome, including nested else branches. Improved path testing ensures better coverage and reduces the risk of logical defects reaching production.